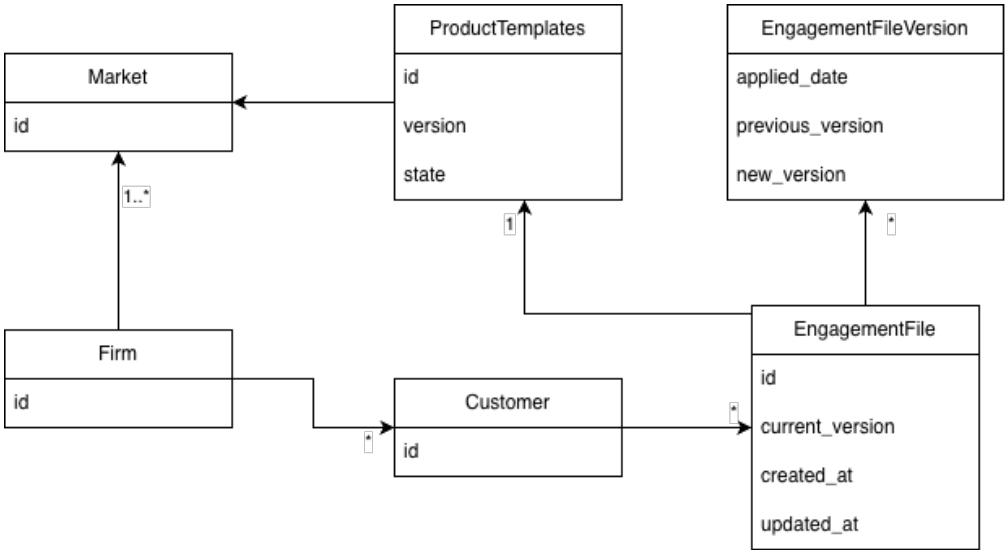
Design Exercise – Engagement Files

By Andres Romero (https://www.linkedin.com/in/anromer/)

Domain Model



This is a simplified domain model, which help to understand the business context, it contains only the relevant entities and properties for the design exercise.

Business Rules

- Engagement files are initially created with the most recent version of product template
- There are multiple updates thought the year
- Customers evaluate their engagement files either apply or decline the update

Constraints

- The stored from of an engagement is not a directly queryable record
- Reading the template information therefore requires loading the engagement, which is a slow operation (~1 minute per file)

Data volumes

- 40 products
- 4.000 firms (Engagement count, median ~40, the largest ~40.000)
- 800.000 active engagement files
- Product templates updates are relatively infrequent, one per week per product

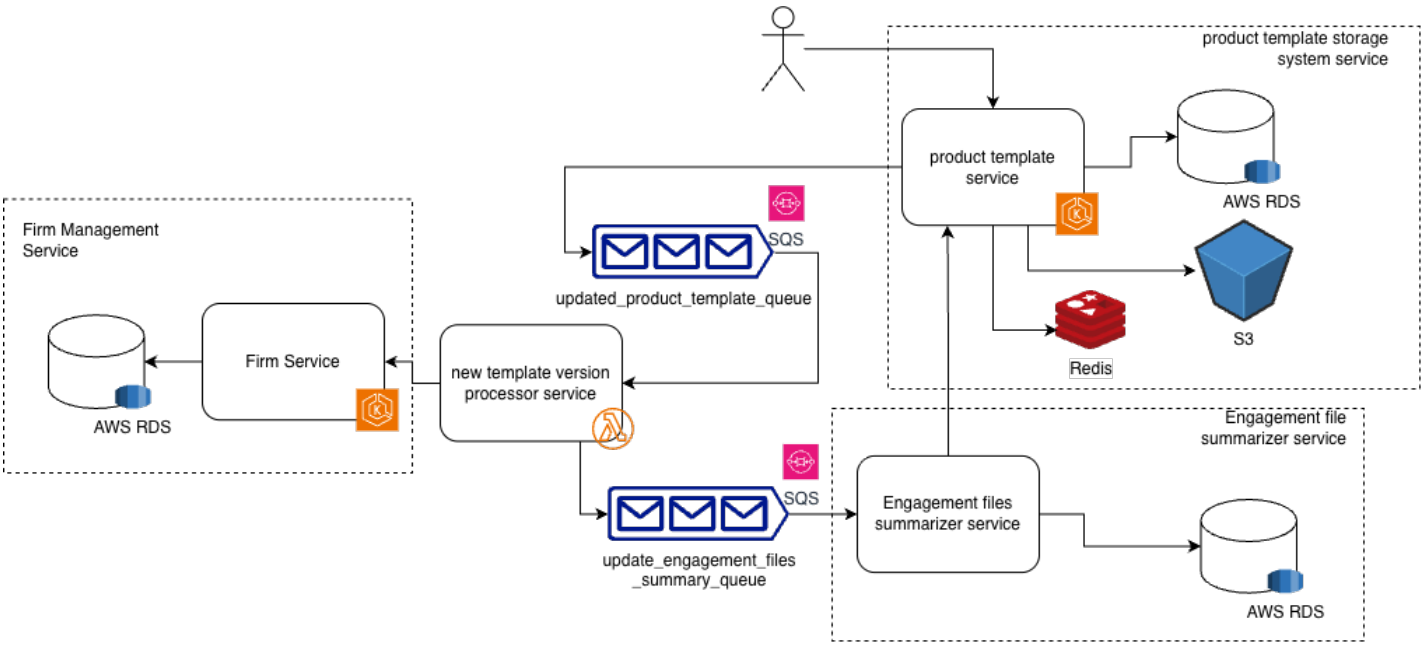
Assumptions

- Clients can have only one version of the product active at a time

Requirements

- See at a glance engagement files with pending updates
- Users needs a summary of pending inbound changes

Solution Design



This approach favor scalability to support more firms and product templates in the future without performance degradation. A microservice architecture with async processing is used for this purpose.

Product Template Storage System Service

This service has the responsibility of store and manages product template information, it stores the product template documents (JSON files) in a **S3 bucket** (folder by product template), and use a relational database (**RDS**) to store information in tables:

- **PRODUCT_TEMPLATE** (id, name, created_at)
- **PRODUCT_TEMPLATE_VERSION** (id, product_template_id, version, state, s3_content_uri, created_at)
- **PRODUCT_TEMPLATE_VERSION_DIFF** (product_template_id, source_version, target_version, diff), a batch process can pre-calculate this difference, this will speed up the process

The API have three main interfaces:

- **Update product template content** (product_template_id , version, content)
- **Create product template** (version, content, market[])
- **Get product template diff** (product_template_id, source_version, target_version)
- **Get product template diff** (product_template_id) it returns all the set of diff between versions

As there are some relevant aspects for this service:

- We should guarantee that only one user is updating a product template at a time; we can create a **temporal lock** in the database to prevent multiple users modifying the same document and let the users know in the UI that this product file is being modified.
- **Idempotency**: It is important guarantee that only request per intention comes to the platform, otherwise we are going to trigger the product update summary process to many times.
- The service which calculates the diff between product template versions could have a **cache**, this will improve the performance

- When a product template is updated or created an **event** will be published to the queue **updated_product_template_queue**, this message store should be ordered. The published event has the following information: product_template_id, new_version, created_at, market[]
- Create and update operations are transactional with the event publish, it is important to maintain the consistency
- Pre-calculate diff between product templates version will speed up the process

Firm Management Service

This service has the responsibility of store and manages the firm information; it is the owner of firm and product template association information. It has its own database with tables like:

- **FIRM** (id, name)
- **CUSTOMER** (id, firm_id, market_id, name)
- **FIRM_CUSTOMER_PRODUCT_TEMPLATE** (firm_id, customer_id, product_template_id, current_version)

The API for this service allow us to query firms and product templates that should be updated when a product template changes. We could have the following interfaces:

- **Get firms by product template** (product_template_id), it returns customers which has this specific product template id, grouped by product template version and by firm. This optimization will reduce a lot the work, given that multiple customer could have the same versions of product templates.

New Template Version Processor Service

This component implement the fan-out pattern for the solution, it pull messages from the **queue updated_product_template_queue,** request firms that are affected by the new version using the **Firm Management Service**, it decides how to split the work and create multiple commands (messages) that will be published to **update_engagement_files_summary_queue**. Architectural features of this service:

- This service could be implemented with a **lambda function**. A dedicate service running all the time is not needed given the frequency of product template updates.
- It deletes the message of the **queue** only when the process has finished successfully. If there is any error, we could use a death letter queue.
- We can implement many kind of algorithms to **split the work**, for instance, for firm with thousands of product templates we can publish multiple command (once per product template or market).
- In this service we can add business rules to define which firm and how product templates processing should be executed.
- Each message published to the topic **update_engagement_files_summary_queue** could have this information: firm_id, product_template_id, current_version, new_available_version, customer_id []

Engagement File Summarizer Service

This component has the logic for creating the summary for new product templates, it also shows the differences between versions. We could configure the infrastructure of this service to make it **elastic**, increase a specific number of instances when there is high load and decrease when the load slowdown. The idea behind this service is to **pre-calculate** the product template summary with the diff between versions. This will make faster the user experience, and we avoid calculate this heavy process each time a user wants to see the summary. Architectural features of this service:

- Given the frequency of product template updates this service could be implemented as a **lambda function** too

- This microservice request diff product template information using the **Product Template Storage System Service**, this information can be **cached** to improve performance during the processing. We can implement a retry and circuit braker approach in case there is any issue with the product template storage system service.
- If there is any error processing the message we can store failed messages in a death letter queue.
- The algorithm of this process should be efficient, it could retrieve all the diff versions for a product template, a single call, then for the each firm information iterate for the product version groping.
- The result of this processing is stored in a database in a bulk insert per firm, we will store only the most recent version. This summary could be stored in the firm management system service, this will simplify the product template apply or decline functionality and update the summary this these operations.
- The table to store the summary is:
    - **FIRM_AVAILABLE_PRODUCT_TEMPLATE_SUMMARY** (firm_id, customer_id, product_template_id, current_version, new_version)

COST calculation

In the worst case scenario the system should process:

1 product template X 40.000 customers X 4.000 firms = 160.000.000 product templates by customer by firm per week

this number is huge, but with this approach we will only insert 1 message in the **updated_product_template_queue** and 4.000 messages in **update_engagement_files_summary_queue.**

This is the free tier. What is expensive are the RDS and the computing services. I will only consider the lambda pricing and discard the RDS pricing, give that these databases are already setup and running for the current systems.

If the processing of a message in the **update_engagement_files_summary_queue** takes less than 2 minutes with a 2GB of memory which cost ~0.002 per invocation, then the pricing for 4000 messages to process is ~8 USD per document template updated per week.